AI Assisted Coding Assignment-11.2

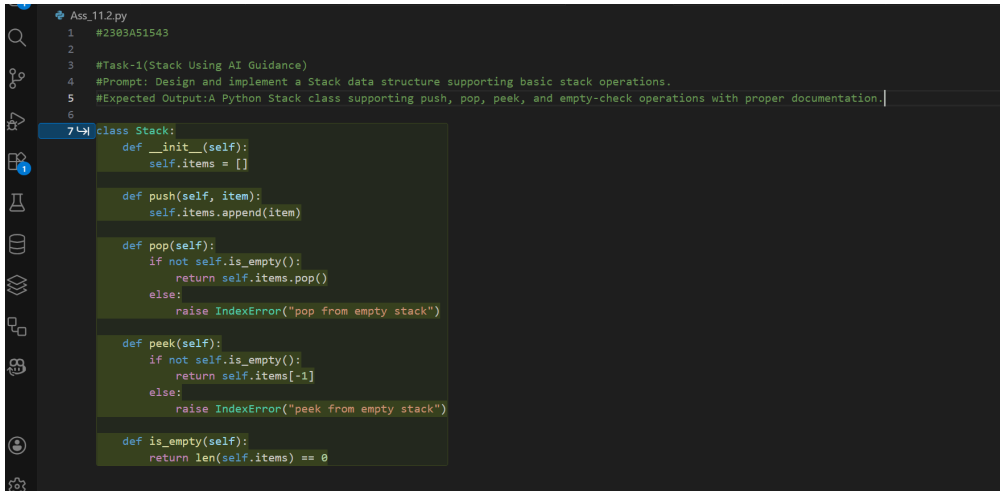
> 2303A51543 BT:29 VELDI.HRUTHIKA 10-3-26

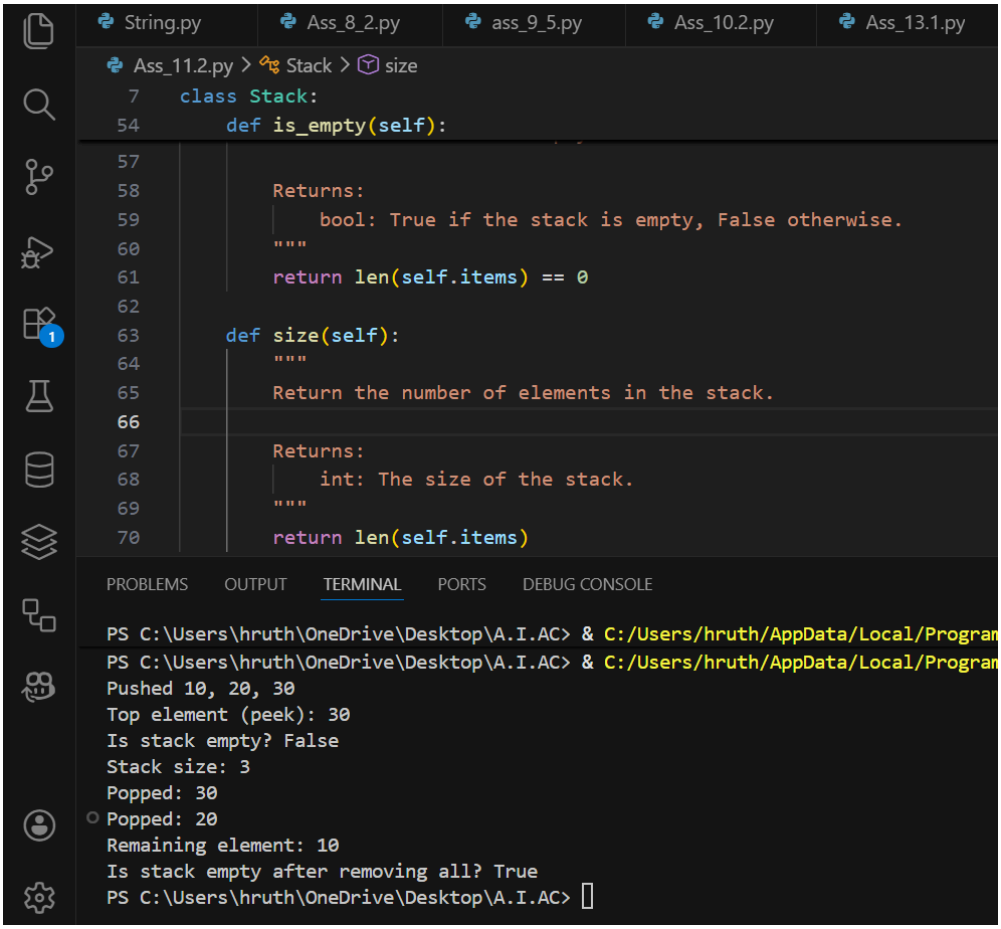
💡 Task Description -1 – (Stack Using AI Guidance)
• Task: With the help of AI, design and implement a Stack data structure supporting basic stack operations.
Expected Output:
• A Python Stack class supporting push, pop, peek, and empty-check operations with proper documentation.

```
#2303A51543

#Task-1(Stack Using AI Guidance)
#Prompt: Design and implement a Stack data structure supporting basic stack operations li
ke pop, push, empty check and keep doc strings in the code.
#Expected Output:A Python Stack class supporting push, pop, peek, and empty-check operati
ons with proper documentation.

class Stack:
    """
    A Stack data structure that implements Last-In-First-Out (LIFO) principle.
    Supports basic operations: push, pop, peek, and empty checking.
    """

    def __init__(self):
        """Initialize an empty stack."""
        self.items = []

    def push(self, item):
        """
        Add an element to the top of the stack.

        Args:
            item: The element to be added to the stack.
        """
        self.items.append(item)

    def pop(self):
        """
        Remove and return the top element from the stack.

        Returns:
            The element at the top of the stack.

        Raises:
            IndexError: If the stack is empty.
        """
```

```
        if self.is_empty():
            raise IndexError("Cannot pop from an empty stack")
        return self.items.pop()

    def peek(self):
        """
        Return the top element of the stack without removing it.

        Returns:
            The element at the top of the stack.

        Raises:
            IndexError: If the stack is empty.
        """
        if self.is_empty():
            raise IndexError("Cannot peek at an empty stack")
        return self.items[-1]

    def is_empty(self):
        """
        Check if the stack is empty.

        Returns:
            bool: True if the stack is empty, False otherwise.
        """
        return len(self.items) == 0

    def size(self):
        """
        Return the number of elements in the stack.

        Returns:
            int: The size of the stack.
        """
        return len(self.items)


# Example usage
if __name__ == "__main__":
    stack = Stack()

    # Test push operation
    stack.push(10)
    stack.push(20)
    stack.push(30)
    print("Pushed 10, 20, 30")

    # Test peek operation
    print(f"Top element (peek): {stack.peek()}")

    # Test is_empty operation
```

```
print(f"Is stack empty? {stack.is_empty()}")

# Test size operation
print(f"Stack size: {stack.size()}")

# Test pop operation
print(f"Popped: {stack.pop()}")
print(f"Popped: {stack.pop()}")

# Test remaining elements
print(f"Remaining element: {stack.peek()}")

# Pop last element
stack.pop()
print(f"Is stack empty after removing all? {stack.is_empty()}")
```





Task Description -2 – (Queue Design)

• Task:Use AI assistance to create a Queue data structure following FIFO principles

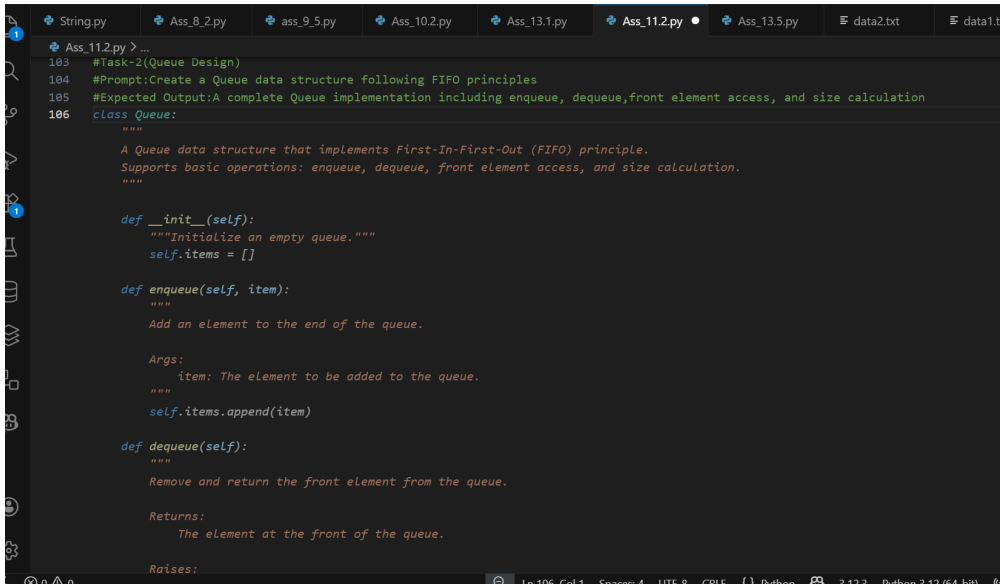
Expected Output:

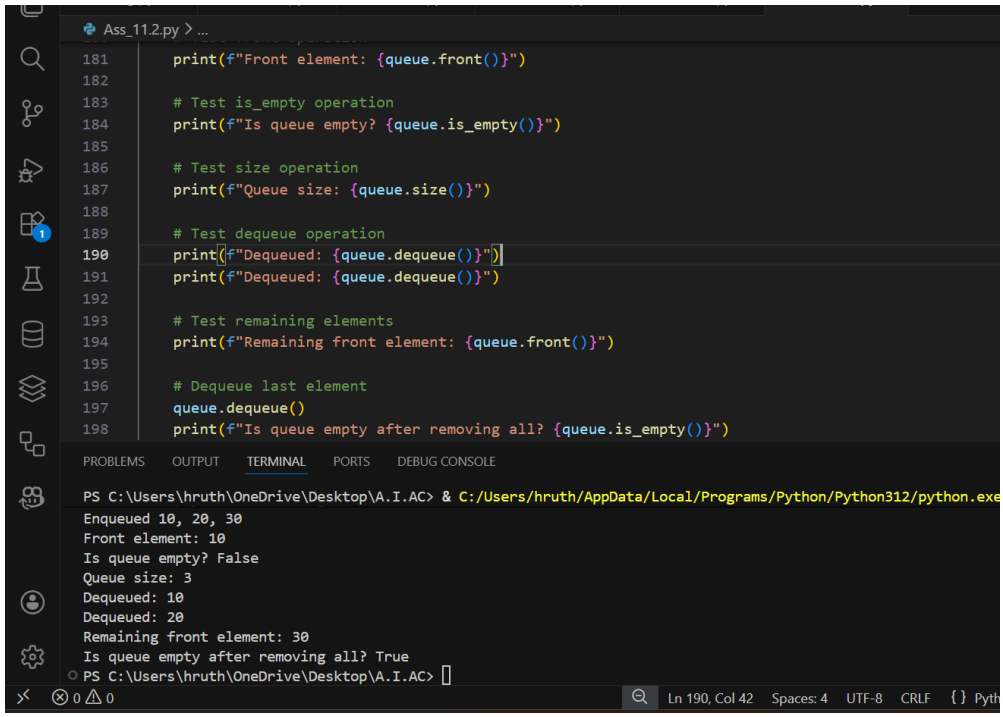
• A complete Queue implementation including enqueue, dequeue, front element access, and size calculation

```
#Task-2(Queue Design)
#Prompt:Create a Queue data structure following FIFO principles
#Expected Output:A complete Queue implementation including enqueue, dequeue,front element access, and size calculation
class Queue:
    """
    A Queue data structure that implements First-In-First-Out (FIFO) principle.
    Supports basic operations: enqueue, dequeue, front element access, and size calculation.
    """

    def __init__(self):
        """Initialize an empty queue."""
        self.items = []
```

```
    def enqueue(self, item):
        """
        Add an element to the end of the queue.

        Args:
            item: The element to be added to the queue.
        """
        self.items.append(item)

    def dequeue(self):
        """
        Remove and return the front element from the queue.

        Returns:
            The element at the front of the queue.

        Raises:
            IndexError: If the queue is empty.
        """
        if self.is_empty():
            raise IndexError("Cannot dequeue from an empty queue")
        return self.items.pop(0)

    def front(self):
        """
        Return the front element of the queue without removing it.

        Returns:
            The element at the front of the queue.

        Raises:
            IndexError: If the queue is empty.
        """
        if self.is_empty():
            raise IndexError("Cannot access front of an empty queue")
        return self.items[0]

    def is_empty(self):
        """
        Check if the queue is empty.

        Returns:
            bool: True if the queue is empty, False otherwise.
        """
        return len(self.items) == 0

    def size(self):
        """
        Return the number of elements in the queue.
```

```
        Returns:
            int: The size of the queue.
        """
        return len(self.items)
# Example usage
if __name__ == "__main__":
    queue = Queue()

    # Test enqueue operation
    queue.enqueue(10)
    queue.enqueue(20)
    queue.enqueue(30)
    print("Enqueued 10, 20, 30")

    # Test front operation
    print(f"Front element: {queue.front()}")

    # Test is_empty operation
    print(f"Is queue empty? {queue.is_empty()}")

    # Test size operation
    print(f"Queue size: {queue.size()}")

    # Test dequeue operation
    print(f"Dequeued: {queue.dequeue()}")
    print(f"Dequeued: {queue.dequeue()}")

    # Test remaining elements
    print(f"Remaining front element: {queue.front()}")

    # Dequeue last element
    queue.dequeue()
    print(f"Is queue empty after removing all? {queue.is_empty()}")
```





💡 Task Description -3 –(Singly Linked List Construction)

• Task: Utilize AI to build a singly linked list supporting insertion and traversal.

Expected Output:

• Correctly functioning linked list with node creation, insertion logic, and display functionality.

```
#Task-3(Singly Linked List Construction)
#Prompt:Build a singly linked list supporting insertion and traversal.
```

```
#Expected Output:Correctly functioning linked list with node creation, insertion,logic, a
nd display functionality.
class Node:
    """
    A Node in a singly linked list.
    Contains data and a reference to the next node.
    """

    def __init__(self, data):
        """Initialize a node with data and no next node."""
        self.data = data
        self.next = None
class SinglyLinkedList:
    """
    A Singly Linked List data structure that supports insertion and traversal.
    """

    def __init__(self):
        """Initialize an empty linked list."""
        self.head = None

    def insert(self, data):
        """
        Insert a new node with the given data at the end of the linked list.

        Args:
            data: The data to be stored in the new node.
        """
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node

    def traverse(self):
        """
        Traverse the linked list and return a list of node data.

        Returns:
            List of data from each node in the linked list.
        """
        elements = []
        current_node = self.head
        while current_node:
            elements.append(current_node.data)
            current_node = current_node.next
        return elements
# Example usage
```

```python
if __name__ == "__main__":
    linked_list = SinglyLinkedList()

    # Test insertion
    linked_list.insert(10)
    linked_list.insert(20)
    linked_list.insert(30)
    print("Inserted 10, 20, 30 into the linked list")

    # Test traversal
    print(f"Linked list elements: {linked_list.traverse()}")
```

```python
#Task-3(Singly Linked List Construction)
#Prompt:Build a singly linked list supporting insertion and traversal.
#Expected Output:Correctly functioning linked list with node creation, insertion,logic, and display functionality.
class Node:
    """
    A Node in a singly linked list.
    Contains data and a reference to the next node.
    """

    def __init__(self, data):
        """Initialize a node with data and no next node."""
        self.data = data
        self.next = None
```

Ass_11.2.py > ...

```python
class SinglyLinkedList:
    def insert(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node

    def traverse(self):
        """
        Traverse the linked list and return a list of node data.

        Returns:
            List of data from each node in the linked list.
        """
        elements = []
        current_node = self.head
        while current_node:
            elements.append(current_node.data)
            current_node = current_node.next
        return elements
```

PROBLEMS OUTPUT TERMINAL PORTS DEBUG CONSOLE

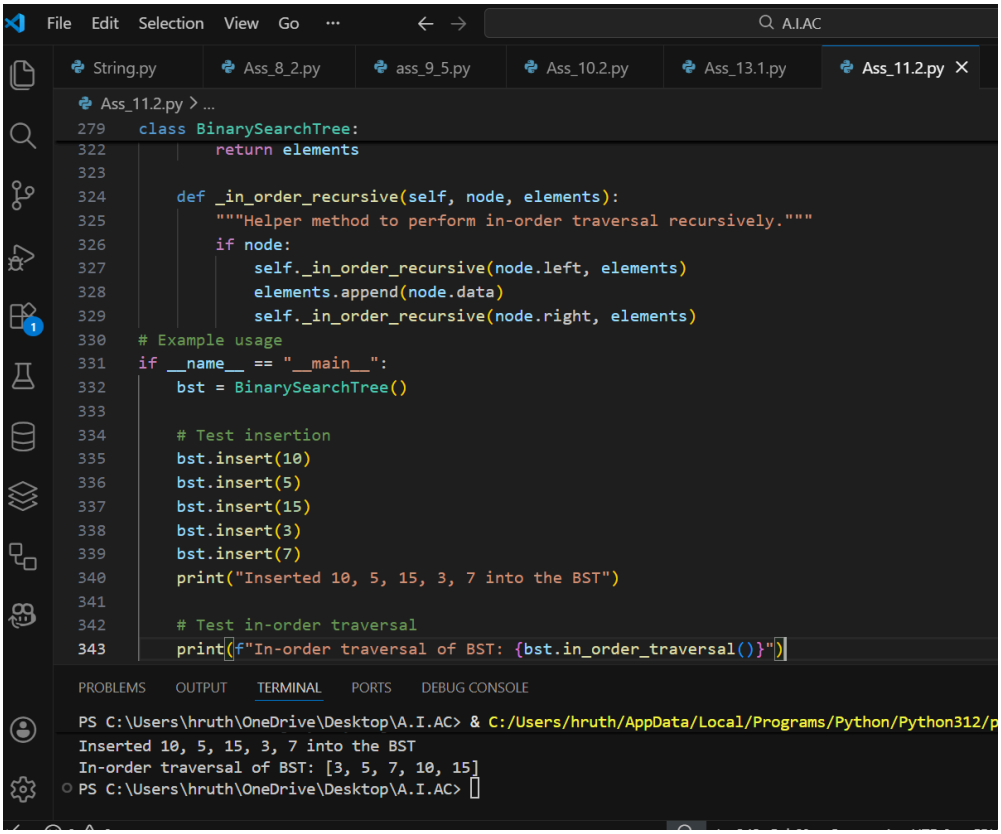
```
Inserted 10, 20, 30 into the linked list
Linked list elements: [10, 20, 30]
PS C:\Users\hruth\OneDrive\Desktop\A.I.AC>
```

💡 Task Description -4 – (Binary Search Tree Operations)
• Task: Implement a Binary Search Tree with AI support focusing on insertion and traversal.
Expected Output:
• BST program with correct node insertion and in-order traversal output.

```
#Task-4(Binary Search Tree Operations)
#Prompt: Implement a Binary Search Tree with AI support focusing on insertion and travers
al.
#Expected Output:BST program with correct node insertion and in-order traversal output.
class TreeNode:
    """
    A node in a Binary Search Tree (BST).
    Contains data and references to left and right child nodes.
    """

    def __init__(self, data):
        """Initialize a tree node with data and no children."""
        self.data = data
        self.left = None
        self.right = None
class BinarySearchTree:
    """
    A Binary Search Tree (BST) data structure that supports insertion and in-order traver
sal.
    """

    def __init__(self):
        """Initialize an empty BST."""
        self.root = None

    def insert(self, data):
        """
        Insert a new node with the given data into the BST.

        Args:
            data: The data to be stored in the new node.
        """
        if self.root is None:
            self.root = TreeNode(data)
        else:
            self._insert_recursive(self.root, data)

    def _insert_recursive(self, node, data):
        """Helper method to insert a node recursively."""
        if data < node.data:
            if node.left is None:
                node.left = TreeNode(data)
```

```python
            else:
                self._insert_recursive(node.left, data)
        else:
            if node.right is None:
                node.right = TreeNode(data)
            else:
                self._insert_recursive(node.right, data)

    def in_order_traversal(self):
        """
        Perform in-order traversal of the BST and return a list of node data.

        Returns:
            List of data from each node in in-order traversal order.
        """
        elements = []
        self._in_order_recursive(self.root, elements)
        return elements

    def _in_order_recursive(self, node, elements):
        """Helper method to perform in-order traversal recursively."""
        if node:
            self._in_order_recursive(node.left, elements)
            elements.append(node.data)
            self._in_order_recursive(node.right, elements)
# Example usage
if __name__ == "__main__":
    bst = BinarySearchTree()

    # Test insertion
    bst.insert(10)
    bst.insert(5)
    bst.insert(15)
    bst.insert(3)
    bst.insert(7)
    print("Inserted 10, 5, 15, 3, 7 into the BST")

    # Test in-order traversal
    print(f"In-order traversal of BST: {bst.in_order_traversal()}")
```

```python
#Task-4(Binary Search Tree Operations)
#Prompt: Implement a Binary Search Tree with AI support focusing on insertion and traversal.
#Expected Output:BST program with correct node insertion and in-order traversal output.
class TreeNode:
    """
    A node in a Binary Search Tree (BST).
    Contains data and references to left and right child nodes.
    """

    def __init__(self, data):
        """Initialize a tree node with data and no children."""
        self.data = data
        self.left = None
        self.right = None
```



💡 Task Description -5 – (Hash Table Implementation)

• Task: Create a hash table using AI with collision handling

Expected Output:

• Hash table supporting insert, search, and delete using chaining or open

```
#Task-5(Hash Table Implementation)
#Prompt: Create a hash table using AI with collision handling
#Expected Output:Hash table supporting insert, search, and delete using chaining or open.
class HashTable:
    """
    A Hash Table data structure that supports insert, search, and delete operations using
chaining for collision handling.
    """

    def __init__(self, size=10):
        """Initialize a hash table with a specified size."""
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        """Generate a hash for the given key."""
        return hash(key) % self.size
```

```
    def insert(self, key, value):
        """
        Insert a key-value pair into the hash table.

        Args:
            key: The key to be inserted.
            value: The value associated with the key.
        """
        index = self._hash(key)
        # Check if the key already exists and update it
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value)
                return
        # If the key does not exist, add a new key-value pair
        self.table[index].append((key, value))

    def search(self, key):
        """
        Search for a value by its key in the hash table.

        Args:
            key: The key to search for.

        Returns:
            The value associated with the key if found, else None.
        """
        index = self._hash(key)
        for k, v in self.table[index]:
            if k == key:
                return v
        return None

    def delete(self, key):
        """
        Delete a key-value pair from the hash table by its key.

        Args:
            key: The key to be deleted.

        Returns:
            True if the deletion was successful, False if the key was not found.
        """
        index = self._hash(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i]
                return True
        return False
# Example usage
if __name__ == "__main__":
```

```
hash_table = HashTable()

# Test insert operation
hash_table.insert("name", "Alice")
hash_table.insert("age", 30)
hash_table.insert("city", "New York")
print("Inserted key-value pairs into the hash table")

# Test search operation
print(f"Search for 'name': {hash_table.search('name')}")
print(f"Search for 'age': {hash_table.search('age')}")
print(f"Search for 'city': {hash_table.search('city')}")

# Test delete operation
print(f"Delete 'age': {hash_table.delete('age')}")
print(f"Search for 'age' after deletion: {hash_table.search('age')}")
```

Ass_11.2.py > ...

```
#Task-5(Hash Table Implementation)
#Prompt: Create a hash table using AI with collision handling
#Expected Output:Hash table supporting insert, search, and delete using chaining or open.
class HashTable:
    """
    A Hash Table data structure that supports insert, search, and delete operations using chaining for collision handling.
    """

    def __init__(self, size=10):
        """Initialize a hash table with a specified size."""
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        """Generate a hash for the given key."""
        return hash(key) % self.size

    def insert(self, key, value):
        """
        Insert a key-value pair into the hash table.

        Args:
            key: The key to be inserted.
            value: The value associated with the key.
        """
        index = self._hash(key)
        # Check if the key already exists and update it
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
```

String.py | Ass_8_2.py | ass_9_5.py | Ass_10.2.py | Ass_13.1.py | Ass_11

Ass_11.2.py > ...

```python
class HashTable:
        return False
# Example usage
if __name__ == "__main__":
    hash_table = HashTable()

    # Test insert operation
    hash_table.insert("name", "Alice")
    hash_table.insert("age", 30)
    hash_table.insert("city", "New York")
    print("Inserted key-value pairs into the hash table")

    # Test search operation
    print(f"Search for 'name': {hash_table.search('name')}")
    print(f"Search for 'age': {hash_table.search('age')}")
    print(f"Search for 'city': {hash_table.search('city')}")

    # Test delete operation
    print(f"Delete 'age': {hash_table.delete('age')}")
    print(f"Search for 'age' after deletion: {hash_table.search('age')}")
```

PROBLEMS OUTPUT TERMINAL PORTS DEBUG CONSOLE

```
PS C:\Users\hruth\OneDrive\Desktop\A.I.AC> & C:/Users/hruth/AppData/Local/Programs/Python/Py
Inserted key-value pairs into the hash table
Search for 'name': Alice
Search for 'age': 30
Search for 'city': New York
Delete 'age': True
Search for 'age' after deletion: None
PS C:\Users\hruth\OneDrive\Desktop\A.I.AC>
```

0 0 Ln 420, Col 5 Spaces: 4